

MVP Report

Joshua Addis, Ashley Corrales, Anthony Nguyen, Vassilios Hapitas, Ivan Muriel-Pacheco

Executive Summary

The Blackjack AI Coach is a Streamlit based MVP that guides and assists users make better decisions by combining implementing expected value analysis into regular decision analysis coaching. The user enters a two card hand and the dealer's upcard, and any additional optional bankroll information, then the app recommends the best action to take: hi, stand double, or split. The goal isn't just to give the user the best move to make, but to explain to the user why that move makes sense in a way that will be remembered and applied in future hands.

The system uses a hybrid architecture, the main recommendation comes from "blackjack_engine.py", which evaluates the player hand, dealer upcard, rule settings, and finite-deck expected values for actions. This makes sure the decision-making layer is grounded in blackjack math instead of relying on a language model to guess the best move. The app then presents the result through three coaching tiers: Table Coach for simple strategy guidance, EV Edge for expected-value comparisons, and Bankroll Desk for risk-aware bankroll framing.

The MVP also includes a nanoGPT-style machine learning workflow designed for explanation support. "build_blackjack_dataset.py" generates training examples for every two-card player hand, every dealer upcard, and each coaching tier. "verify_two_card_coverage.py" checks that the dataset is complete, readable, and encodable before training. "train.py", "model.py", and "sample.py" provide the training and inference pipeline, while the Streamlit app can use grounded responses from the training data or a trained checkpoint as an explanation layer.

Overall, the MVP demonstrates the core idea of an AI-assisted blackjack decision system: to use reliable expected-value logic to choose the best action, then use an AI-style language layer to make the recommendation easier for users to understand. The current version focuses on blackjack rather than the full blackjack-and-poker scope from the original proposal, which keeps the project realistic while still proving the main technical concept. Future improvements could expand beyond two-card training coverage, improve the language model output, add more casino rule variations, and revisit poker probability modeling.

User & Use Case

Primary User Persona

The primary user for this MVP is a beginner to intermediate blackjack player who wants to improve and level up their game with a personal coach at their fingertips. Most casual blackjack players can apply basic hit, stand, and betting strategies with their own hand, but struggle to apply a wider net and analysis across the whole table in real time. Traditional strategies require card counting and memorizing each player's hand, but that has a huge learning curve not to mention illegal in most casinos. Players (users) need a simple system that gives immediate explanations and recommendations in real time that is easy to understand and won't have them kicked from the casino.

Core Use case

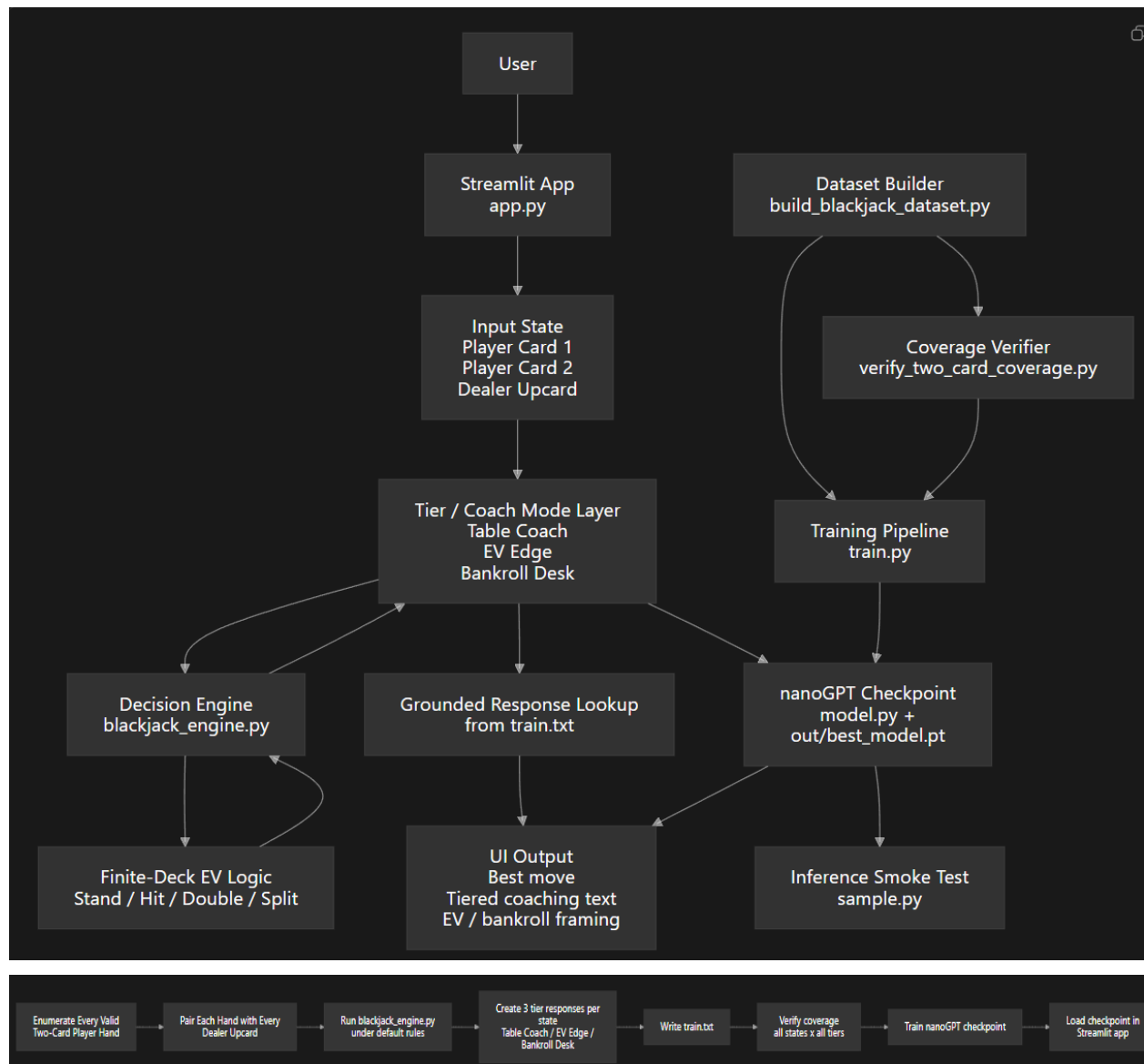
This MVP is designed as a blackjack training and decision-support tool. It has its own data that contains two full decks of cards and a training model it runs through which revolutionizes the blackjack game removing the need of ever counting cards on the live table. The user enters their hand, the dealer upcard, one of the three coach options, and the allowed rules on the table such as whether doubling or splitting is allowed. The system then analyzes the hand and returns the best action, let it be hit, stand, double, or split, with a corresponding confidence score and explanation of the play. In more advanced settings, the app can also show expected-value comparisons and bankroll-oriented guidance. In this advanced Bankroll desk coach you put in your current bank size and then how much you want to bet on your current hand, and the MVP will give you the breakdown on the math on how much you can potentially win or lose.

Why this MVP fits this user

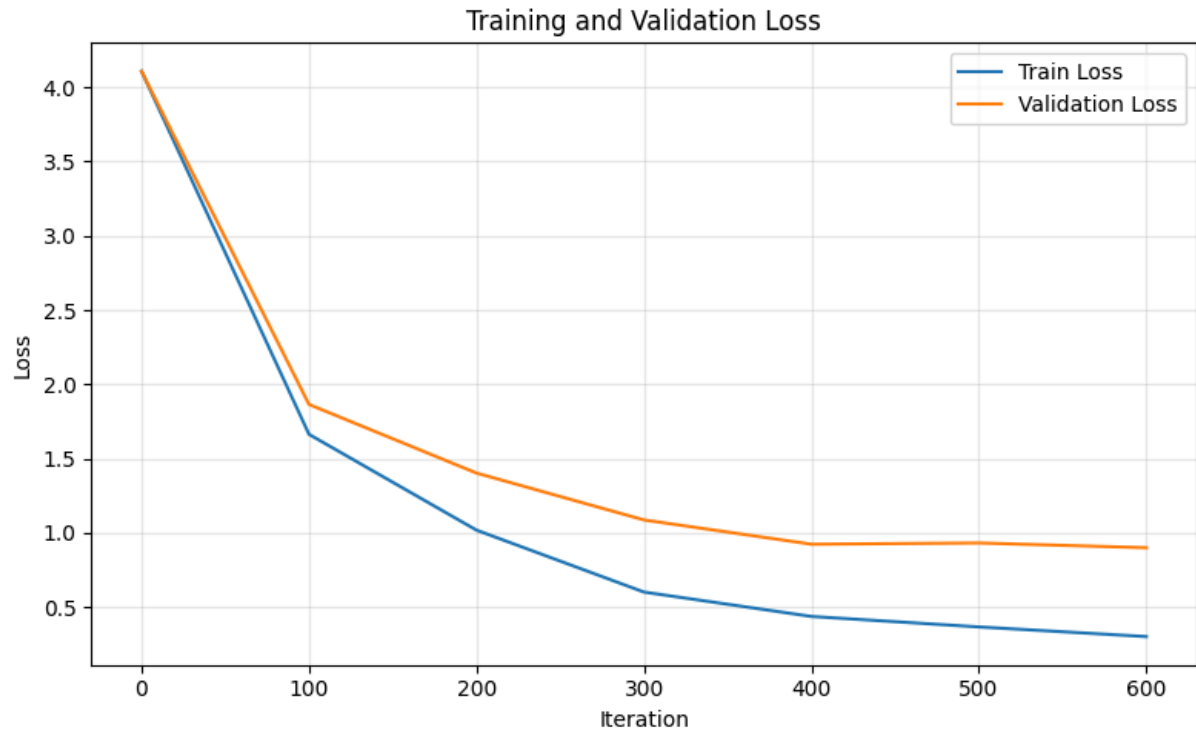
This persona fits the MVP well because the current product's strongest tool is its coach mode. Its value comes from combining a rules-based blackjack engine with readable explanations, making it user friendly towards all players. This works for students, self-leaners, and semi-professional players who want guided practice rather than full casino-risk training.

System Design

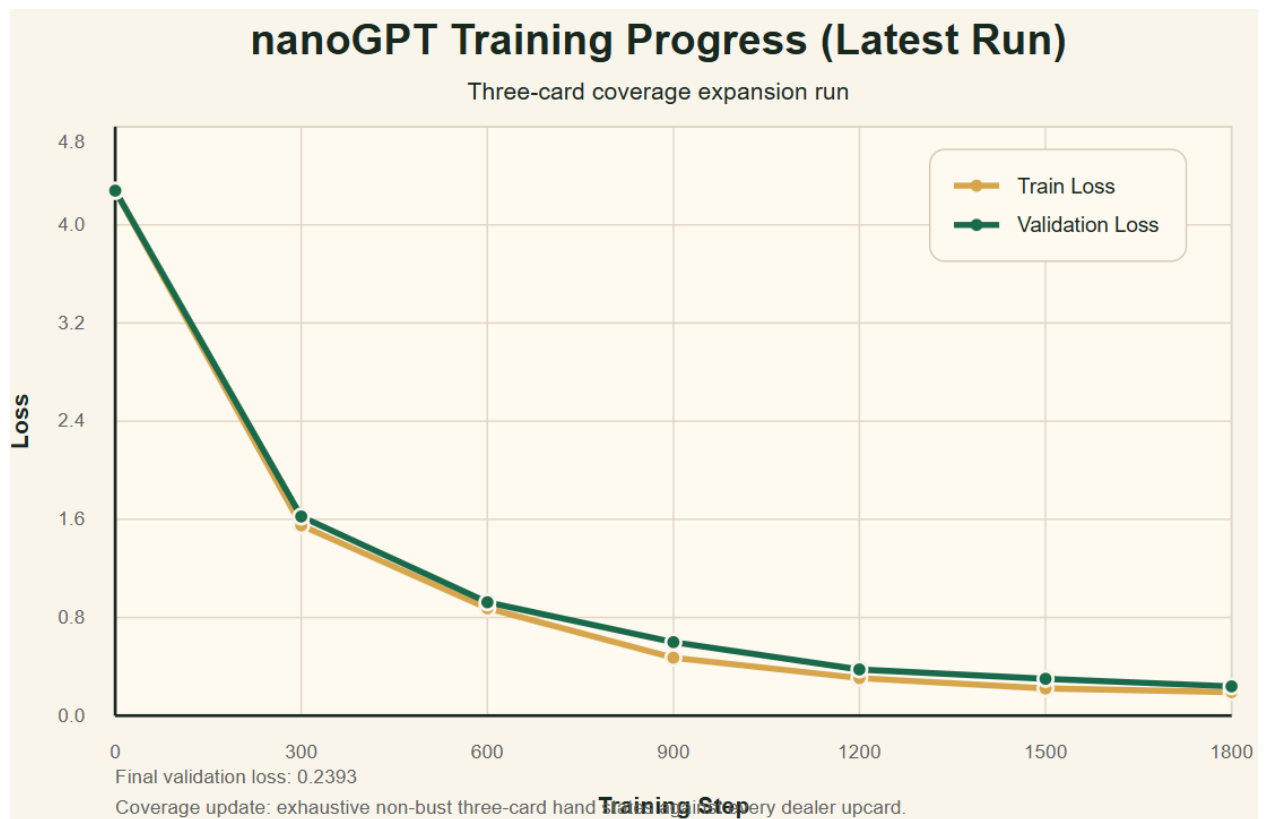
Our user interface is built with the Streamlit app, which runs a blackjack engine written in Python. We structured three levels of product outputs. The first tier is the Table Coach, which offers a very basic explanation. Tier two is our Expected Value Edge, which provides a more mathematical analysis with the sole purpose of staying true to disciplined blackjack play that offers the player insight into the long run. Tier three is our Bankroll Desk, which tracks the total amount of money you have and the bet size to give you a better insight into your mathematical edge over the dealer.



These images show the flow of our system and how we trained the NanoGPT.



This is the validation graph after the first training session for the NanoGPT.



This is the second training run, which achieved a validation loss of about 0.2393. With a significant boost in our NanoGPT’s ability to generate text for cases that have two player cards and one dealer card. Below is the command window from our first training session, and the difference between the NanoGPT’s output for a “trained hand” and a “not trained hand.” After a few more training sessions, we got a NanoGPT that produces consistent outputs for simple blackjack hand combinations.

```
C:\Users\Ivan>cd "C:\Users\Ivan\OneDrive\Documents\BJ Coach 301\Project-301--Implemented-BJ-Coach\mvp"

C:\Users\Ivan\OneDrive\Documents\BJ Coach 301\Project-301--Implemented-BJ-Coach\mvp>"C:\Users\Ivan\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" build_blackjack_dataset.py
Building opening-hand states...
Collected 48 sampled opening-hand states after 49 attempts.
Building sampled later-hit states...
Collected 12 sampled later-hand states for 3-card player hands after 22 attempts.
Collected 12 sampled later-hand states for 4-card player hands after 46 attempts.
Prepared 72 unique serialized examples.
Wrote 72 training examples to C:\Users\Ivan\OneDrive\Documents\BJ Coach 301\Project-301--Implemented-BJ-Coach\mvp\data\train.txt

C:\Users\Ivan\OneDrive\Documents\BJ Coach 301\Project-301--Implemented-BJ-Coach\mvp>"C:\Users\Ivan\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" train.py
Dataset Length: 18950 characters
Train length: 17055
Val length: 1895
Vocab size: 58
Device: cpu
CPU quick-start mode enabled: using smaller batches and fewer iterations.
Model parameters: 880,768
step    0 | train loss 4.1082 | val loss 4.1069 | time 0.8s
Saved new best checkpoint at step 0

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | e|6k|7|Kbn1{fob|f,e,Jftu7y6jnTxDve
-wJj90{f7k4t5}
xHBdPabbA6ooP01F53PQ}vvFx88ahKj.PPaHQQneKSF'sse:T8JJRn4eclt6kvvfcESS58
--- End Sample ---

step   100 | train loss 1.6632 | val loss 1.8644 | time 10.9s
Saved new best checkpoint at step 100

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | -0.1} -0.4han: hagaiousto 15 | | Pan: | Reason: | | Re | -14 Dead | EVsot 0.2} | {' : | Pa, | | -0.5714 Rtalag: EVs:
--- End Sample ---

step   200 | train loss 1.0181 | val loss 1.4024 | time 20.9s
Saved new best checkpoint at step 200

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | | Paioutak: Desestale : h, | | EV: 0 | {'Hit': -0.2,0.4, 2336 -0.8 | EVs: -0.4 'Hitan: -0.4 Fan: | EV: EVs: {'Stand'Dou
--- End Sample ---

step   300 | train loss 0.6011 | val loss 1.0867 | time 31.0s
Saved new best checkpoint at step 300

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | Soft: False | Pair: | Palse An: Stand De: | Stand Stand EV: -0K EV: {' : {' : -0.1380.532, 'Hitand': -0.3686} | | Reason:
--- End Sample ---

step   400 | train loss 0.4370 | val loss 0.9233 | time 41.1s
Saved new best checkpoint at step 400

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | Total: 10 | Soft: False | Pair: False | Action: Hit | EV: -0.23 | {'Double': 0.17, 'Hit': ' : -0.5544, 'Stand': -0.1498}
--- End Sample ---

step   500 | train loss 0.3664 | val loss 0.9317 | time 52.2s

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | Total: 15 | Soft: False | Pair: False | Action: Best EV: -0.0820 | EVs: {'Hit': -1.16} 'Stand': 0.3014} | Reason: A har
--- End Sample ---

step   600 | train loss 0.3014 | val loss 0.8997 | time 63.8s
Saved new best checkpoint at step 600

--- Sample Generation ---
Player: 8,8 | Dealer: 6 | Total: 15 | Soft: False | Pair: False | Action: Hit | Best EV: -0.2168 | EVs: {'Double': -0.1542, 'Double': -0.4015, 'H
--- End Sample ---

Training complete.
Best validation loss: 0.8997
```

NANO GPT COACH VOICE

Explanation-only language layer

Cachasechalll: Coach Mathath stand ving oon othever m dealeag lestoostlly ooo vir tom discing ttttooon dienes ceces ecy

NANO GPT COACH VOICE

Retrieval-grounded language layer

Split is the clean table play here. Teaching tip: pair strategy is about whether the two cards are stronger together or as two new starting hands.

Data

The MVP uses a synthetically generated dataset derived from existing blackjack rules and probability theory, which was created programmatically using a custom script (`build_blackjack_dataset.py`) that lists valid game states and labels them using a decision engine. This ensures that each data point reflects optimal decision-making based on expected value, aligning with the project's goal of building an AI system rooted from probabilistic reasoning rather than observational data.

The dataset was constructed by generating all valid combinations of player hands and dealer upcards, while enforcing realistic constraints such as a two-deck limit and removing invalid or bust states. Each state is also represented in chronological order to avoid duplication and is labeled with the optimal action (hit, stand, double, split) computed from expected value calculations. Beyond simple labels, each example also includes detailed expected value outputs for all possible actions, reasoning for the chosen move, and multiple explanation styles (ex. Bankroll focused, expected value focused, and instructional “table coach” perspectives).

The data is serialized into a structured text format for GPT-style training, where each example maps a game state to both an optimal decision and a simple/comprehensive language explanation. This allows the model to learn not only what action to take, but also how to explain that decision to a user, which is critical for the MVP's coaching functionality. The dataset provides broad coverage of the blackjack decision space, including all starting hands and extended states after hits, along with all possible dealer upcards. Preprocessing includes formatting, tokenization, and splitting into training and validation sets.

Models

The MVP uses a GPT-style language model to learn both optimal blackjack decisions and the reasoning behind them. Unlike a traditional classifier that maps inputs directly to action, our model is designed as a decision and explanation system, leveraging the structured, text-based dataset to generate both the correct move and a comprehensive justification.

The model takes as input a serialized representation of the game state, including the player's hand, dealer upcard, and relevant conditions. It is trained to predict outputs that include the optimal action, expected value metrics, and explanatory text, such as reasoning and coaching insights. However, because the training data includes multiple explanation styles (ex. Expected value perspective, bankroll management, and instructional coaching), the model learns to produce responses that are not only correct but also interpretable and useful for users.

Architecturally, the model follows a simplified transformer-based design inspired by nanoGPT, consisting of token embeddings, positional encodings, stacked self-attention blocks, and feedforward layers. Training is performed using next-token prediction on the serialized dataset, allowing the model to learn the sequential structure of both decision logic and natural language explanations. This approach enables the system to generalize across different blackjack states while maintaining coherent and context-aware outputs.

The model also complements a rule-based decision engine used during dataset generation. While the engine provided mathematically optimal labels based on expected value, the language model learns to approximate this behavior and present it in a flexible, user-friendly format. This hybrid design allows the MVP to function as both a decision support tool and an AI coach, which is capable of explaining strategy in real time.

Overall, the model transforms structured probabilistic data into an interactive experience, enabling users to understand not just what action to take, but why that action is the most optimal in their scenario.

Evaluation

Here's a list of all our Quantitative metrics in numerical data:

Primary language: Python

Core Python files: 6

Total Python code size: 2,549 lines

UI layer: [app.py](#) file

Decision engine: [blackjack_engine.py](#) file

ML/data pipeline: [model.py](#), [train.py](#), [build_blackjack_dataset.py](#), and [sample.py](#)

External dependencies: listed in requirements.text (streamlit, torch, matplotlib)

Supported player actions: 4 (Hit, Stand, Double, Split)

Coaching/product tiers: 3 (Table Coach, EV Edge, Bankroll desk)

Coaching modes: 3

Dataset generator coverage: opening hands plus later-hit states up to 5 player cards

Training configuration: block size (128) , layers (4), attention heads (4), embedding size (128), batch size (32), max iterations (3000), learning rate (3e-4)

Model size formula: $793,344 + 128 \times \text{vocab_size}$ with trainable parameters in file [train.py](#)

Qualitative Assessment

Our MVP is one of the strongest in the class for multiple reasons. One it completes a full product loop; user input, decision logic, explanation layer, and working interface. We are especially strong in nanoGPT support analysis and realtime feedback. It doesn't just output a move; it explains why the move is recommended, where to improve on, what your projected winnings might be, which makes the Ai feel more like a coach than a simple calculator.

Another one of our major strengths is the hybrid architecture. The blackjack decision making is handled by deterministic logic in our [blackjack_engine.py](#) file, while the GPT-style model explains it all for you. This smart design uses traditional logic which is strongly suited for when accuracy matters most and uses the GPT model to turn outputs into your natural-language.

The app has three tiers and three coaching modes to help tailor to multiple skill levels and game types with a live website that you can access at any time. The main weakness is on our end where the machine realizes heavily on already set data and checkpoint outputs, making it harder

to teach it further on a game that may have three decks, more than two players, or a different currency than USD. Also, the engine is still an MVP rather than a full- casino grade simulator, which is also its plus since it doesn't rely on counting cards, and doesn't break any casino rules.

Overall our product is more mature as an explainable blackjack coaching MVP than as a fully validated ML(machine learning) system, which is reasonable for a first version prototype of its kind. It won't act like Chat-GPT where it learns patterns, that is still in the works, but it's a huge step forward into the future of cards.

Limitations & Risks

(Failure modes, biases, data issues, privacy concerns.)

This app is a blackjack coaching prototype, not a casino-grade simulator or financial advisory tool. Its most reliable recommendations come from a rule-based expected value engine, while the nanoGPT model is mainly used to explain and present those decisions. As a result, the move logic should be treated as more dependable than the generated coaching text. The language layer can help users understand a recommendation, but it should not be viewed as an independent source of truth.

The project also has important scope limitations. It is strongest in the standard blackjack setup of two player cards and one dealer upcard, where dataset coverage and verification are most complete. Although the system can handle some more complex hand states, those cases are more likely to rely on fallback explanations generated from the engine. The app also assumes a simplified rules environment and does not account for every casino variation, such as surrender, insurance, resplitting rules, or split-ace restrictions. Since blackjack strategy is sensitive to table rules, recommendations may not always match optimal play in a real casino.

The nanoGPT component introduces additional risk. Even when trained on the project dataset, a small character-level model may produce awkward, repetitive, or overly confident wording. The app reduces this risk through grounded retrieval and engine-backed fallbacks, but raw generated text should still be treated cautiously. The dataset is synthetic and generated by the project's own engine, meaning the model learns to restate the engine's assumptions. Any simplification or blind spot in the engine can therefore carry into the training data.

Bias in this project is primarily modeling and framing bias rather than demographic bias. The app favors expected-value reasoning, disciplined play, and bankroll restraint. This is intentional, but it may underrepresent real-world factors such as fatigue, emotional tilt, table speed, or personal financial stress. Some coaching tiers, especially "Bankroll Desk," may sound authoritative even though the system does not know the user's full gambling context.

Overall, this system is best understood as an MVP decision-support and education tool. It is useful for demonstrating structured blackjack guidance, but it should not be treated as a fully validated commercial gambling engine or a substitute for professional financial judgment.

Next Steps

With more time, this project has the potential to grow into something much grander than a simple blackjack coach. This model, as previously stated, is mostly a demonstration of the potential of using artificial intelligence and nanoGPT technology in order to assist one of the most well known and simple card games of chance there is. However, given 2-3 more months of work on the project, the Blackjack AI Coach could evolve to not only be the best blackjack coach on the market, but could very well also branch out into other casino and card games.

Technical

Having more time to work on the AI Coach, the first of many next steps would be to lessen as many of the limitations of the current model as possible. This would include the ability to add custom “table rules” to the model. Since there is a high likelihood of house rules overlapping between casinos, adding some of the most common table rules would increase the usability of the Blackjack AI Coach.

Another good addition to the model would be a “Comparison” feature, likely included in the higher membership tiers. This feature would take the best option given by the coach, and compare it with the other possible actions for the situation given. The model, and subsequently the nanoGPT would then explain how the other options provide a smaller chance of a positive outcome compared to the recommended play.

Product:

With more time, the model could possibly be expanded to include other simple casino card games, such as 3 card poker. It is important to note that these newer game models would not be nearly as fleshed out and error-free as the blackjack model; however they would be a good start for the future of this product. The membership tiers could give members early access to game models that are currently being worked on. This would help greatly with bug-testing, and would also give more of a reward to loyal customers than simply a better blackjack explanation.

An additional possibility for the future of this Blackjack AI Coach as a product (most likely for the highest tier), would be to let the user play practice blackjack games against the coach. This would run similarly to chess bots, where the AI would give feedback *after* the player’s turn, in order to train the player to be able to be proficient at the game through *active* learning. Due to the high likelihood of casinos banning the use of this product in their buildings, this option would allow for further use of the Blackjack AI Coach despite the restrictions.

In conclusion, the next best steps, given time, would be to minimize the limitations and failure chances of the current model, while also expanding the product's use for the purpose of longer and more engaged use.